

ASSIGNMENT NO. 12

MINI PROJECT

TRAINING INSTITUTE MANAGEMENT SYSTEM

First Name: Isha Lad

Class: S.Y.B.Sc. Data Science Sem - 3

Roll No.: 20978

Project Topic: Database design, ER modeling, relational schema, table creation, sample data, relationships, and business analysis for a Training Institute.

1. Assignment Overview

The objective of Assignment 12 is to develop a basic MySQL Training Institute Management System for managing students, trainers, courses, batches, enrollments, attendance, and assessment results. The project demonstrates database planning, table relationships, query writing, business analysis, and presentation of findings.

1.1 Main Objectives

- Identify the main entities required by a training institute.
- Design an ER diagram showing the relationships among the entities.
- Convert the conceptual design into a relational schema with primary and foreign keys.
- Create the database and all required tables in MySQL.
- Insert realistic sample records into every table.
- Verify foreign-key relationships and retrieve information using SQL queries.
- Generate useful reports such as enrollment, attendance, assessment, ranking, and progress reports.

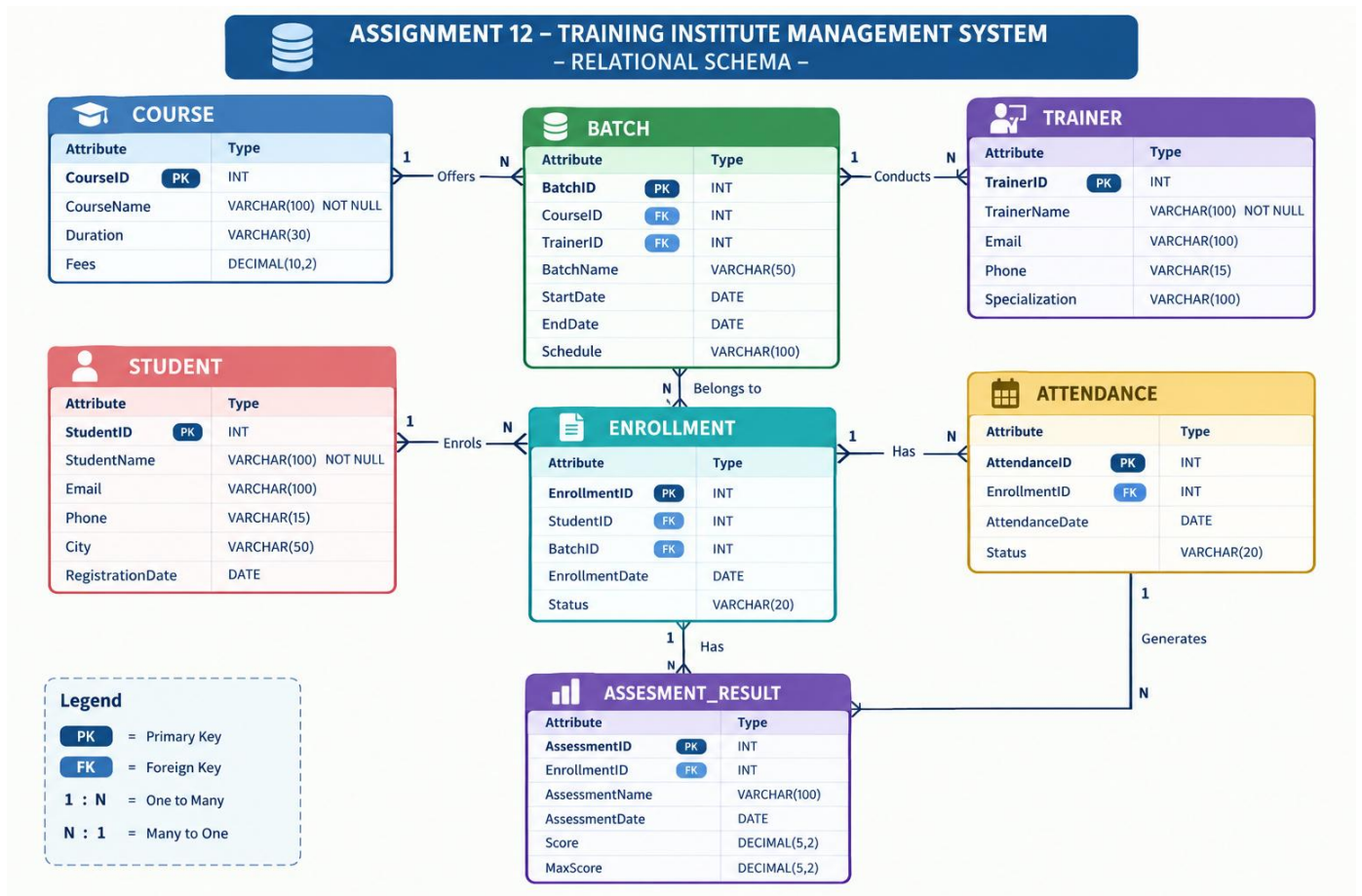
1.2 Required Entities

No.	Entity / Table	Purpose
1	Student	Stores student details.
2	Trainer	Stores trainer details.
3	Course	Stores course information.
4	Batch	Stores course batch and trainer assignment.
5	Enrollment	Connects students with batches.

6	Attendance	Stores attendance records for enrollments.
7	AssessmentResult	Stores assessment scores for enrollments.

2. Relational Schema Diagram

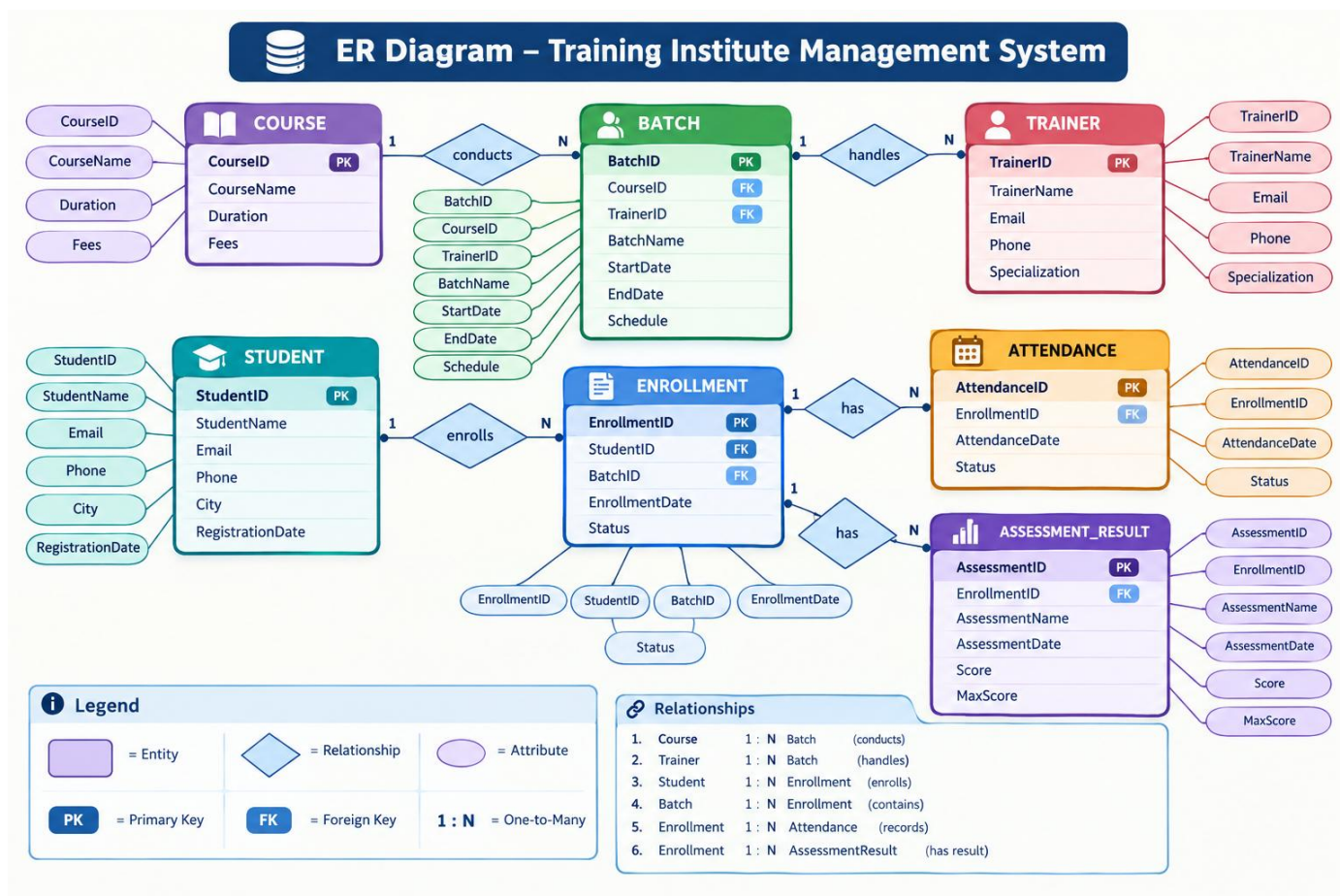
The relational schema represents the seven tables, their attributes, primary keys (PK), foreign keys (FK), and one-to-many relationships. The diagram is designed to satisfy the relationship requirements given in the workbook.



PK = Primary Key FK = Foreign Key 1:N = One-to-Many relationship

3. ER Diagram – Standard ER Notation

The ER diagram is the conceptual design. Rectangles represent entities, ovals represent attributes, diamonds represent relationships, and 1:N labels represent cardinality.



Required relationships: Course–Batch (1:N), Trainer–Batch (1:N), Student–Enrollment (1:N), Batch–Enrollment (1:N), Enrollment–Attendance (1:N), Enrollment–AssessmentResult (1:N).

4. Table Structure / Data Dictionary

Student

Column	Data Type	Constraint
StudentID	INT	PK
StudentName	VARCHAR(100)	NOT NULL
Email	VARCHAR(100)	NOT NULL, UNIQUE
Phone	VARCHAR(15)	NOT NULL, UNIQUE
City	VARCHAR(50)	NOT NULL
RegistrationDate	DATE	NOT NULL, DEFAULT (CURRENT_DATE)

Trainer

Column	Data Type	Constraint
TrainerID	INT	PK
TrainerName	VARCHAR(100)	NOT NULL
Email	VARCHAR(100)	NOT NULL, UNIQUE
Phone	VARCHAR(15)	NOT NULL, UNIQUE

Specialization	VARCHAR(100)	NOT NULL
----------------	--------------	----------

Course

Column	Data Type	Constraint
CourseID	INT	PK
CourseName	VARCHAR(100)	NOT NULL
Duration	VARCHAR(30)	NOT NULL
Fees	DECIMAL(10,2)	NOT NULL, CHECK (Fees >= 0)

Batch

Column	Data Type	Constraint
BatchID	INT	PK
CourseID	INT	FK → Course
TrainerID	INT	FK → Trainer
BatchName	VARCHAR(50)	NOT NULL, UNIQUE
StartDate	DATE	NOT NULL
EndDate	DATE	NOT NULL, CHECK (EndDate >= StartDate)
Schedule	VARCHAR(100)	NOT NULL

Enrollment

Column	Data Type	Constraint
EnrollmentID	INT	PK
Column	Data Type	Constraint
StudentID	INT	FK → Student
BatchID	INT	FK → Batch
EnrollmentDate	DATE	NOT NULL, DEFAULT (CURRENT_DATE)
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Active', CHECK (Status IN ('Active', 'Completed', 'Cancelled'))

Attendance

Column	Data Type	Constraint
AttendanceID	INT	PK
EnrollmentID	INT	FK → Enrollment
AttendanceDate	DATE	NOT NULL
Status	VARCHAR(20)	NOT NULL, CHECK (Status IN ('Present', 'Absent', 'Late'))

AssessmentResult

Column	Data Type	Constraint
AssessmentID	INT	PK
EnrollmentID	INT	FK → Enrollment
AssessmentName	VARCHAR(100)	NOT NULL

AssessmentDate	DATE	NOT NULL
Score	DECIMAL(5,2)	NOT NULL, CHECK (Score >= 0)
MaxScore	DECIMAL(5,2)	NOT NULL, CHECK (MaxScore > 0)

5. MySQL Code – Complete Database Creation

```
CREATE DATABASE TrainingInstituteDB;
USE TrainingInstituteDB;
```

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(100) NOT NULL,
    Email VARCHAR(100),
    Phone VARCHAR(15),
    City VARCHAR(50),
    RegistrationDate DATE
);
```

The screenshot shows the phpMyAdmin interface. On the left is a sidebar with a tree view of databases and tables. The main area displays the 'AssessmentResult' table in the 'seniorcollegedb' database. The table has 5 rows of data. The columns are: StudentID, StudentName, DateOfBirth, Gender, Email, Phone, and Address. The data is as follows:

StudentID	StudentName	DateOfBirth	Gender	Email	Phone	Address
1	Aarav Sharma	2004-05-12	Male	aarav@gmail.com	9876543210	Pune
2	Priya Patil	2003-08-21	Female	priya@gmail.com	9876543211	Pune
3	Rahul Kulkarni	2004-01-15	Male	rahul@gmail.com	9876543212	Mumbai
4	Sneha Joshi	2003-11-10	Female	sneha@gmail.com	9876543213	Nashik
5	Rohan Deshmukh	2004-03-18	Male	rohan@gmail.com	9876543214	Pune

```
\CREATE TABLE Trainer (
    TrainerID INT PRIMARY KEY,
    TrainerName VARCHAR(100) NOT NULL,
    Email VARCHAR(100),
    Phone VARCHAR(15),
    Specialization VARCHAR(100)
);
```

phpMyAdmin

Recent Favorites

Database: seniorcollegedb Table: AssessmentResult

Bookmark this SQL query

Showing rows 0 - 2 (3 total, Query took 0.0007 seconds.)

SELECT * FROM Trainer;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	TrainerID	TrainerName	Email	Phone	Specialization	JoiningDate
☐ Edit ☐ Copy ☐ Delete	1	Amit Verma	amit@gmail.com	9000000001	Python and Data Science	2023-06-10
☐ Edit ☐ Copy ☐ Delete	2	Neha Sharma	neha@gmail.com	9000000002	SQL and Database	2022-08-15
☐ Edit ☐ Copy ☐ Delete	3	Vikas Patil	vikas@gmail.com	9000000003	Web Development	2024-01-20

Check all | With selected: Edit Copy Delete Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Console

```
CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100) NOT NULL,
    Duration VARCHAR(30),
    Fees DECIMAL(10,2)
);
```

phpMyAdmin

Recent Favorites

Database: seniorcollegedb Table: AssessmentResult

Bookmark this SQL query

Showing rows 0 - 3 (4 total, Query took 0.0017 seconds.)

SELECT * FROM Course;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	CourseID	CourseName	DurationMonths	Description
☐ Edit ☐ Copy ☐ Delete	1	Python Programming	6	Complete Python programming course
☐ Edit ☐ Copy ☐ Delete	2	Data Science	8	Data Science with Python and Machine Learning
☐ Edit ☐ Copy ☐ Delete	3	Web Development	6	HTML, CSS, JavaScript and Backend Development
☐ Edit ☐ Copy ☐ Delete	4	Database Management	4	MySQL and Database Management

Check all | With selected: Edit Copy Delete Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Console

```
CREATE TABLE Batch (
    BatchID INT PRIMARY KEY,
    CourseID INT NOT NULL,
    TrainerID INT NOT NULL,
    BatchName VARCHAR(50),
    StartDate DATE,
    EndDate DATE,
    Schedule VARCHAR(100),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID),
    FOREIGN KEY (TrainerID) REFERENCES Trainer(TrainerID)
);
```


phpMyAdmin

Server: 127.0.0.1 » Database: seniorcollegedb » Table: AssessmentResult

Showing rows 0 - 3 (4 total, Query took 0.0017 seconds.)

SELECT * FROM Batch;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	BatchID	CourseID	TrainerID	BatchName	StartDate	EndDate	Schedule
☐ Edit ☐ Copy ☐ Delete	1	1	1	Python Batch A	2026-01-10	2026-07-10	10:00 AM
☐ Edit ☐ Copy ☐ Delete	2	2	2	1 Data Science Batch A	2026-02-01	2026-10-01	02:00 PM
☐ Edit ☐ Copy ☐ Delete	3	3	3	Web Development Batch A	2026-01-15	2026-07-15	11:00 AM
☐ Edit ☐ Copy ☐ Delete	4	4	4	2 Database Batch A	2026-03-01	2026-07-01	03:00 PM

Check all | With selected: Edit Copy Delete Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Console

```
CREATE TABLE Enrollment (
    EnrollmentID INT PRIMARY KEY,
    StudentID INT NOT NULL,
    BatchID INT NOT NULL,
    EnrollmentDate DATE,
    Status VARCHAR(20),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (BatchID) REFERENCES Batch(BatchID)
);
```

phpMyAdmin

Server: 127.0.0.1 » Database: seniorcollegedb » Table: AssessmentResult

Showing rows 0 - 4 (5 total, Query took 0.0013 seconds.)

SELECT * FROM Enrollment;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	EnrollmentID	StudentID	BatchID	EnrollDate	Status
☐ Edit ☐ Copy ☐ Delete	1	1	1	2026-01-05	Active
☐ Edit ☐ Copy ☐ Delete	2	2	1	2026-01-06	Active
☐ Edit ☐ Copy ☐ Delete	3	3	2	2026-01-25	Active
☐ Edit ☐ Copy ☐ Delete	4	4	3	2026-01-10	Completed
☐ Edit ☐ Copy ☐ Delete	5	5	4	2026-02-20	Active

Check all | With selected: Edit Copy Delete Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Console Copy to clipboard Export Display chart Create view

```
CREATE TABLE Attendance (
    AttendanceID INT PRIMARY KEY,
    EnrollmentID INT NOT NULL,
    AttendanceDate DATE,
    Status VARCHAR(20),
    FOREIGN KEY (EnrollmentID) REFERENCES Enrollment(EnrollmentID)
);
```

phpMyAdmin

Server: 127.0.0.1 » Database: seniorcollegedb » Table: AssessmentResult

SELECT * FROM Attendance;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	AttendanceID	EnrollmentID	AttendanceDate	Status	Remarks
☐ Edit ☐ Copy ☐ Delete	1	1	2026-08-01	Present	Regular attendance
☐ Edit ☐ Copy ☐ Delete	2	1	2026-08-02	Present	Regular attendance
☐ Edit ☐ Copy ☐ Delete	3	1	2026-08-03	Late	Arrived 10 minutes late
☐ Edit ☐ Copy ☐ Delete	4	2	2026-08-01	Absent	Personal reason
☐ Edit ☐ Copy ☐ Delete	5	2	2026-08-02	Present	Regular attendance
☐ Edit ☐ Copy ☐ Delete	6	2	2026-08-03	Present	Regular attendance
☐ Edit ☐ Copy ☐ Delete	7	3	2026-08-01	Present	Regular attendance
☐ Edit ☐ Copy ☐ Delete	8	3	2026-08-02	Present	Regular attendance
☐ Edit ☐ Copy ☐ Delete	9	5	2026-08-01	Present	Regular attendance
☐ Edit ☐ Copy ☐ Delete	10	5	2026-08-02	Late	Traffic issue

Check all | With selected: Edit Copy Delete Export

```
CREATE TABLE AssessmentResult (
    AssessmentID INT PRIMARY KEY,
    EnrollmentID INT NOT NULL,
    AssessmentName VARCHAR(100),
    AssessmentDate DATE,
    Score DECIMAL(5,2),
    MaxScore DECIMAL(5,2),
    FOREIGN KEY (EnrollmentID) REFERENCES Enrollment(EnrollmentID)
);
```

phpMyAdmin

Server: 127.0.0.1 » Database: seniorcollegedb » Table: AssessmentResult

SELECT * FROM AssessmentResult;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

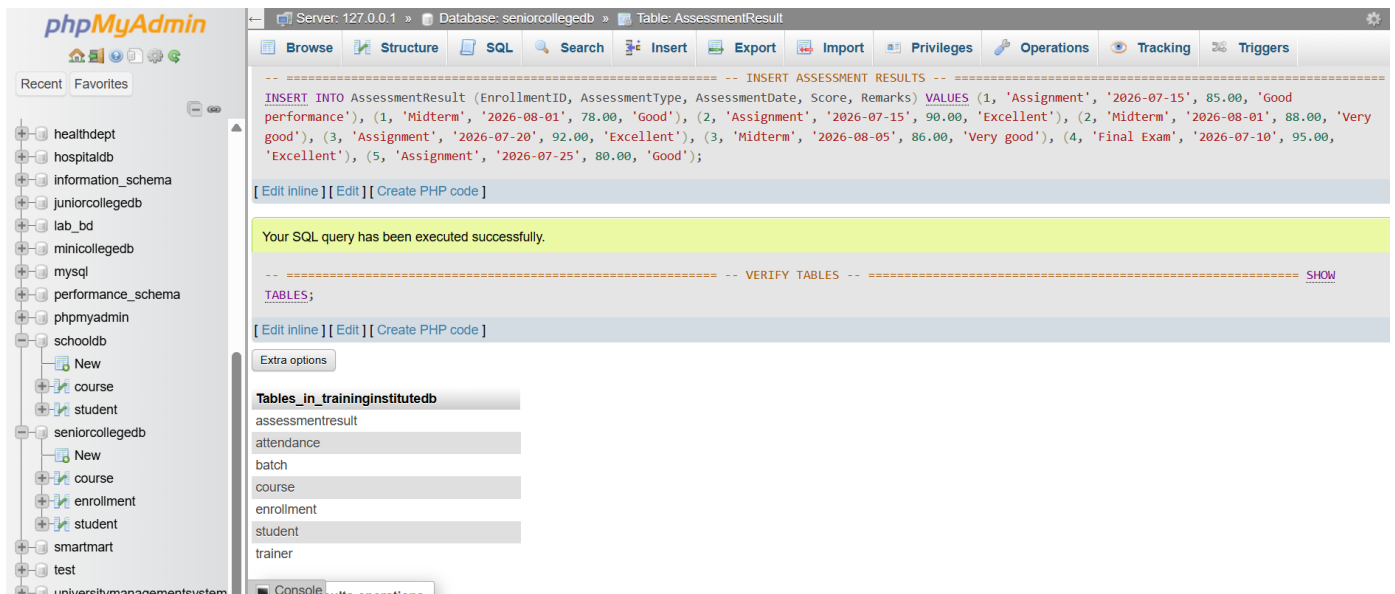
Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	AssessmentID	EnrollmentID	AssessmentType	AssessmentDate	Score	Remarks
☐ Edit ☐ Copy ☐ Delete	1	1	Assignment	2026-07-15	85.00	Good perform
☐ Edit ☐ Copy ☐ Delete	2	1	Midterm	2026-08-01	78.00	Good
☐ Edit ☐ Copy ☐ Delete	3	2	Assignment	2026-07-15	90.00	Excellent
☐ Edit ☐ Copy ☐ Delete	4	2	Midterm	2026-08-01	88.00	Very good
☐ Edit ☐ Copy ☐ Delete	5	3	Assignment	2026-07-20	92.00	Excellent
☐ Edit ☐ Copy ☐ Delete	6	3	Midterm	2026-08-05	86.00	Very good
☐ Edit ☐ Copy ☐ Delete	7	4	Final Exam	2026-07-10	95.00	Excellent
☐ Edit ☐ Copy ☐ Delete	8	5	Assignment	2026-07-25	80.00	Good

Check all | With selected: Edit Copy Delete Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None



6. MySQL Code – Sample Data

```

INSERT INTO Course VALUES
(101,'Python for Data Science','3 Months',15000.00),
(102,'Full Stack Web Development','6 Months',30000.00),
(103,'Data Analytics with SQL','3 Months',18000.00);

INSERT INTO Trainer VALUES
(1,'Amit Sharma','amit@institute.com','9876500001','Python & Data Science'),
(2,'Neha Patil','neha@institute.com','9876500002','Web Development'),
(3,'Rahul Joshi','rahul@institute.com','9876500003','SQL & Analytics');

INSERT INTO Student VALUES
(1,'Sujata Mali','sujata@gmail.com','9876500011','Pune','2026-07-01'),
(2,'Rahul Deshmukh','rahul@gmail.com','9876500012','Mumbai','2026-07-02'),
(3,'Priya Patil','priya@gmail.com','9876500013','Pune','2026-07-03'),
(4,'Aman Verma','aman@gmail.com','9876500014','Nashik','2026-07-04'),
(5,'Sneha Joshi','sneha@gmail.com','9876500015','Thane','2026-07-05');

INSERT INTO Batch VALUES
(201,101,1,'PY-DS-01','2026-07-10','2026-10-10','Mon-Wed-Fri 10 AM'),
(202,102,2,'WEB-01','2026-07-15','2027-01-15','Tue-Thu-Sat 11 AM'),
(203,103,3,'SQL-01','2026-07-20','2026-10-20','Mon-Wed 2 PM');

INSERT INTO Enrollment VALUES
(301,1,201,'2026-07-08','Active'),
(302,2,201,'2026-07-08','Active'),
(303,3,202,'2026-07-12','Active'),
(304,4,203,'2026-07-18','Active'),
(305,5,201,'2026-07-09','Active');

INSERT INTO Attendance VALUES
(401,301,'2026-07-11','Present'),
(402,301,'2026-07-13','Present'),
(403,302,'2026-07-11','Absent'),
(404,302,'2026-07-13','Present'),
(405,303,'2026-07-16','Present'),
(406,304,'2026-07-21','Present'),
(407,305,'2026-07-11','Present');

INSERT INTO AssessmentResult VALUES
(501,301,'Unit Test 1','2026-08-01',88,100),
(502,302,'Unit Test 1','2026-08-01',76,100),
(503,303,'Unit Test 1','2026-08-05',91,100),
(504,304,'Unit Test 1','2026-08-07',69,100),
(505,305,'Unit Test 1','2026-08-01',84,100);

```

7. Business Queries and Data Analysis

1. Students enrolled in each batch

```

SELECT b.BatchName, s.StudentName
FROM Enrollment e
JOIN Student s ON e.StudentID = s.StudentID
JOIN Batch b ON e.BatchID = b.BatchID

```

ORDER BY b.BatchName;

The screenshot shows the phpMyAdmin interface with the 'traininginstitutedb' database selected. The 'Query' tab is active, displaying the following SQL query:

```
>SELECT b.BatchName, s.StudentName FROM Enrollment e JOIN Student s ON e.StudentID = s.StudentID JOIN Batch b ON e.BatchID = b.BatchID
```

The query results are displayed in a table with the following data:

BatchName	StudentName
Data Science Batch A	Rahul Kulkarni
Database Batch A	Rohan Deshmukh
Python Batch A	Aarav Sharma
Python Batch A	Priya Patil
Web Development Batch A	Sneha Joshi

Explanation: Uses JOIN to connect students, enrollments, and batches.

2. Course and trainer assigned to every batch

```
SELECT b.BatchName, c.CourseName, t.TrainerName
FROM Batch b
JOIN Course c ON b.CourseID = c.CourseID
JOIN Trainer t ON b.TrainerID = t.TrainerID;
```

The screenshot shows the phpMyAdmin interface with the 'traininginstitutedb' database selected. The 'Query' tab is active, displaying the following SQL query:

```
>SELECT b.BatchName, s.StudentName FROM Enrollment e JOIN Student s ON e.StudentID = s.StudentID JOIN Batch b ON e.BatchID = b.BatchID
```

The query results are displayed in a table with the following data:

BatchName	CourseName	TrainerName
Python Batch A	Python Programming	Amit Verma
Data Science Batch A	Data Science	Amit Verma
Web Development Batch A	Web Development	Vikas Patil
Database Batch A	Database Management	Neha Sharma

Explanation: Shows the course and trainer associated with each batch.

3. Total students in each course

```
SELECT c.CourseName, COUNT(e.StudentID) AS TotalStudents
FROM Course c
LEFT JOIN Batch b ON c.CourseID = b.CourseID
LEFT JOIN Enrollment e ON b.BatchID = e.BatchID
GROUP BY c.CourseID, c.CourseName;
```

Server: 127.0.0.1 » Database: traininginstitutedb » Table: Course

Extra options

CourseName	TotalStudents
Python Programming	2
Data Science	1
Web Development	1
Database Management	1

Query results operations

Console

```
Press Ctrl+Enter to execute query
> SELECT b.BatchName, s.StudentName FROM Enrollment e JOIN Student s ON e.StudentID = s.StudentID JOIN Batch b ON e.BatchID = b.BatchID
> -- 2. Course and trainer assigned to every batch SELECT b.BatchName, c.CourseName, t.TrainerName FROM Batch b JOIN Course c ON b.CourseID = c.CourseID
> SELECT c.CourseName, COUNT(e.StudentID) AS TotalStudents FROM Course c LEFT JOIN Batch b ON c.CourseID = b.CourseID
```

Explanation: COUNT and GROUP BY calculate course-wise enrollment.

4. Batches with more than a specified number of students

```
SELECT b.BatchName, COUNT(e.EnrollmentID) AS TotalStudents
FROM Batch b
JOIN Enrollment e ON b.BatchID = e.BatchID
GROUP BY b.BatchID, b.BatchName
HAVING COUNT(e.EnrollmentID) > 2;
```

Explanation: HAVING filters grouped results after COUNT.

5. Average assessment score for each batch

```
SELECT b.BatchName, AVG(a.Score) AS AverageScore
FROM Batch b
JOIN Enrollment e ON b.BatchID = e.BatchID
JOIN AssessmentResult a ON e.EnrollmentID = a.EnrollmentID
GROUP BY b.BatchID, b.BatchName;
```

Server: 127.0.0.1 » Database: traininginstitutedb » Table: Batch

Extra options

BatchName	AverageScore
Python Batch A	85.250000
Data Science Batch A	89.000000
Web Development Batch A	95.000000
Database Batch A	80.000000

Query results operations

Console

```
> SELECT b.BatchName, COUNT(e.EnrollmentID) AS TotalStudents FROM Batch b JOIN Enrollment e ON b.BatchID = e.BatchID GROUP BY b.BatchID, b.BatchName
> SELECT * FROM Enrollment:
```

Explanation: AVG gives the average assessment score for each batch.

6. Rank students according to assessment results(Windows Function)

```
SELECT s.StudentName, a.Score,
       RANK() OVER (ORDER BY a.Score DESC) AS StudentRank
FROM AssessmentResult a
JOIN Enrollment e ON a.EnrollmentID = e.EnrollmentID
```

JOIN Student s ON e.StudentID = s.StudentID;

StudentName	Score	StudentRank
Sneha Joshi	95.00	1
Rahul Kulkarni	92.00	2
Priya Patil	90.00	3
Priya Patil	88.00	4
Rahul Kulkarni	86.00	5
Aarav Sharma	85.00	6
Rohan Deshmukh	80.00	7
Aarav Sharma	78.00	8

Console:

```
> SELECT b.BatchName, COUNT(e.EnrollmentID) AS TotalStudents FROM Batch b JOIN Enrollment e ON b.BatchID = e.BatchID GROUP BY b.BatchName;
> SELECT * FROM Enrollment;
```

Explanation: RANK() is a window function that assigns rank based on score.

7. Student-wise training progress report

```
SELECT s.StudentName, b.BatchName,
       COUNT(DISTINCT at.AttendanceID) AS AttendanceRecords,
       AVG(ar.Score) AS AverageScore
FROM Student s
JOIN Enrollment e ON s.StudentID = e.StudentID
JOIN Batch b ON e.BatchID = b.BatchID
LEFT JOIN Attendance at ON e.EnrollmentID = at.EnrollmentID
LEFT JOIN AssessmentResult ar ON e.EnrollmentID = ar.EnrollmentID
GROUP BY s.StudentID, s.StudentName, b.BatchID, b.BatchName;
```

StudentName	BatchName	AttendanceRecords	AverageScore
Aarav Sharma	Python Batch A	3	81.500000
Priya Patil	Python Batch A	3	89.000000
Rahul Kulkarni	Data Science Batch A	2	89.000000
Sneha Joshi	Web Development Batch A	0	95.000000
Rohan Deshmukh	Database Batch A	2	80.000000

Query results operations:

Print Copy to clipboard Export Display chart Create view

Console:

```
> SELECT b.BatchName, COUNT(e.EnrollmentID) AS TotalStudents FROM Batch b JOIN Enrollment e ON b.BatchID = e.BatchID GROUP BY b.BatchName;
> SELECT * FROM Enrollment;
```

Explanation: Combines attendance and assessment information into one progress report.

Query 8 — Students whose attendance is below required percentage:

For example, using 75% as the required attendance:

SELECT

s.StudentName,

ROUND(

100 * SUM(CASE WHEN a.Status = 'Present' THEN 1 ELSE 0 END)

```

        / COUNT(a.AttendanceID), 2
    ) AS AttendancePercentage
FROM Student s
JOIN Enrollment e
    ON s.StudentID = e.StudentID
JOIN Attendance a
    ON e.EnrollmentID = a.EnrollmentID
GROUP BY s.StudentID, s.StudentName
HAVING AttendancePercentage < 75;

```

Server: 127.0.0.1 » Database: traininginstitutedb » Table: Student

Number of rows: 25 | Filter rows: Search this table | Sort by key: None

StudentName	AttendancePercentage
Aarav Sharma	66.67
Priya Patil	66.67
Rohan Deshmukh	50.00

Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Console

```

> SELECT b.BatchName, COUNT(e.EnrollmentID) AS TotalStudents FROM Batch b JOIN Enrollment e ON b.BatchID = e.BatchID GRO...
> SELECT b.BatchName, AVG(a.Score) AS AverageScore FROM Batch b JOIN Enrollment e ON b.BatchID = e.BatchID JOIN Assessme...

```

Query 9 — Highest-performing student in every batch:

```

SELECT BatchName, StudentName, Score
FROM (
    SELECT
        b.BatchName,
        s.StudentName,
        a.Score,
        RANK() OVER (
            PARTITION BY b.BatchID
            ORDER BY a.Score DESC
        ) AS rnk
    FROM AssessmentResult a
    JOIN Enrollment e
        ON a.EnrollmentID = e.EnrollmentID
    JOIN Student s
        ON e.StudentID = s.StudentID

```

```

JOIN Batch b
ON e.BatchID = b.BatchID
) x
WHERE rnk = 1;

```

The screenshot shows the phpMyAdmin interface for a database named 'traininginstitutedb'. The 'Student' table is selected, and its structure is displayed. The table has three columns: 'BatchName', 'StudentName', and 'Score'. The query results show four rows of data:

BatchName	StudentName	Score
Python Batch A	Priya Patil	90.00
Data Science Batch A	Rahul Kulkarni	92.00
Web Development Batch A	Sneha Joshi	95.00
Database Batch A	Rohan Deshmukh	80.00

The console at the bottom shows the following SQL queries:

```

> SELECT b.BatchName, COUNT(e.EnrollmentID) AS TotalStudents FROM Batch b JOIN Enrollment e ON b.BatchID = e.BatchID GRO...
> SELECT b.BatchName, AVG(a.Score) AS AverageScore FROM Batch b JOIN Enrollment e ON b.BatchID = e.BatchID JOIN Assesseme...

```

Query 10 — Students who have not attended any session:

```

SELECT
s.StudentName
FROM Student s
JOIN Enrollment e
ON s.StudentID = e.StudentID
LEFT JOIN Attendance a
ON e.EnrollmentID = a.EnrollmentID
WHERE a.AttendanceID IS NULL;

```

The screenshot shows the phpMyAdmin interface for the same database. The 'Enrollment' table is selected, and its structure is displayed. The table has two columns: 'StudentName' and 'EnrollmentID'. The query results show one row of data:

StudentName	EnrollmentID
Sneha Joshi	

The console at the bottom shows the following SQL query:

```

e.EnrollmentID = a.EnrollmentID WHERE a.AttendanceID IS NULL;

```


Query 11 — Course-wise student performance:

SELECT

```
c.CourseName,  
COUNT(DISTINCT e.StudentID) AS TotalStudents,  
ROUND(AVG(ar.Score), 2) AS AverageScore
```

FROM Course c

JOIN Batch b

```
ON c.CourseID = b.CourseID
```

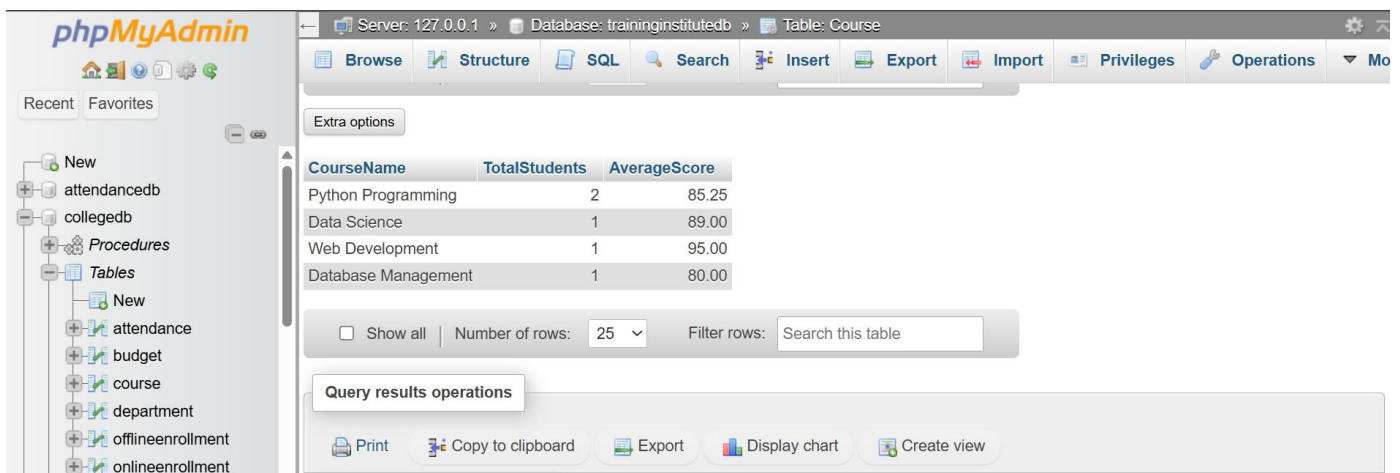
JOIN Enrollment e

```
ON b.BatchID = e.BatchID
```

JOIN AssessmentResult ar

```
ON e.EnrollmentID = ar.EnrollmentID
```

GROUP BY c.CourseID, c.CourseName;



The screenshot shows the phpMyAdmin interface. The left sidebar displays a tree view of the database structure, including 'attendance', 'budget', 'course', 'department', 'offlineenrollment', and 'onlineenrollment'. The main content area shows the results of a query for the 'Course' table. The table has three columns: 'CourseName', 'TotalStudents', and 'AverageScore'. The data is as follows:

CourseName	TotalStudents	AverageScore
Python Programming	2	85.25
Data Science	1	89.00
Web Development	1	95.00
Database Management	1	80.00

Below the table, there are options to 'Show all', 'Number of rows: 25', and a 'Filter rows' search box. At the bottom, there are buttons for 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'.

Query 12 — Highest and lowest average attendance batch:

SELECT

```
b.BatchName,  
ROUND(  
100 * SUM(CASE WHEN a.Status = 'Present' THEN 1 ELSE 0 END)  
/ COUNT(a.AttendanceID), 2  
) AS AverageAttendance
```

FROM Batch b

JOIN Enrollment e

```
ON b.BatchID = e.BatchID
```

JOIN Attendance a

```

ON e.EnrollmentID = a.EnrollmentID
GROUP BY b.BatchID, b.BatchName
ORDER BY AverageAttendance DESC;

```

The screenshot shows the phpMyAdmin interface for a database named 'traininginstitutedb'. The 'Batch' table is selected, and its structure is displayed. The table has two columns: 'BatchName' and 'AverageAttendance'. The query results show three rows of data.

BatchName	AverageAttendance
Data Science Batch A	100.00
Python Batch A	66.67
Database Batch A	50.00

The console at the bottom shows the following SQL query:

```

> SELECT b.BatchName, COUNT(e.EnrollmentID) AS TotalStudents FROM Batch b JOIN Enrollment e ON b.BatchID = e.BatchID GRO...

```

Query 13 — Use a VIEW for progress:

```
CREATE VIEW StudentProgress AS
```

```
SELECT
```

```

    s.StudentName,
    b.BatchName,
    COUNT(DISTINCT a.AttendanceID) AS AttendanceRecords,
    ROUND(AVG(ar.Score), 2) AS AverageScore

```

```
FROM Student s
```

```
JOIN Enrollment e
```

```
    ON s.StudentID = e.StudentID
```

```
JOIN Batch b
```

```
    ON e.BatchID = b.BatchID
```

```
LEFT JOIN Attendance a
```

```
    ON e.EnrollmentID = a.EnrollmentID
```

```
LEFT JOIN AssessmentResult ar
```

```
    ON e.EnrollmentID = ar.EnrollmentID
```

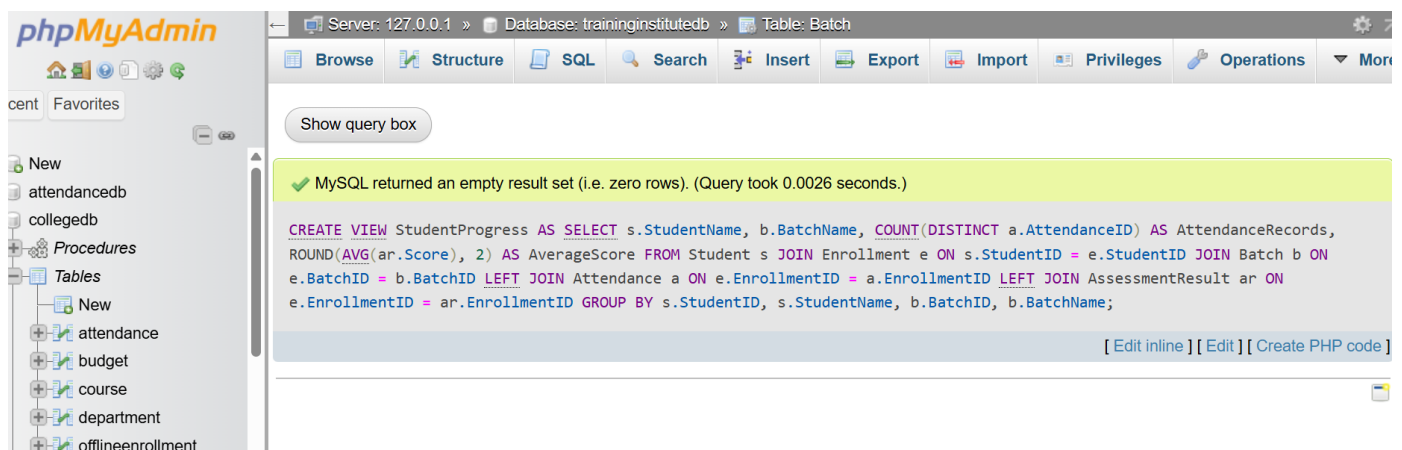
```
GROUP BY
```

```

    s.StudentID,
    s.StudentName,

```

b.BatchID,
b.BatchName;



10. Insights and Findings

You can paste this:

1. Highest Student Enrollment Course

The course with the highest number of enrolled students can be identified using the course-wise enrollment query. This helps the institute understand which courses have greater demand and plan batches and trainers accordingly.

2. Low Attendance Batches

Batches with lower attendance percentages can be identified using attendance analysis. The institute can follow up with students, identify attendance issues, and take corrective action.

3. High-Performing Students

Students with the highest assessment scores can be identified using ranking and batch-wise performance queries. These students can be recognized and encouraged to maintain their performance.

4. Low-Performing Students

Students with lower assessment scores can be identified from the assessment results. Trainers can provide additional practice, guidance, and individual support to improve their performance.

5. Improvement Suggestions

The institute can use the analysis to improve training by monitoring attendance regularly, providing additional support to low-performing students, recognizing high-performing students, and planning courses and batches according to enrollment demand.

This fits the assignment's expected insights, which include course-wise enrollment, low-attendance batches, high/low performers, student progress and improving training planning/monitoring.

8. Relationship Explanation

Relationship	Type	Meaning	Implementation
Course → Batch	1:N	One course can be conducted through multiple batches.	CourseID in Batch is FK.
Trainer → Batch	1:N	One trainer can handle multiple batches.	TrainerID in Batch is FK.
Student → Enrollment	1:N	One student can have multiple enrollments.	StudentID in Enrollment is FK.
Batch → Enrollment	1:N	One batch can contain multiple student enrollments.	BatchID in Enrollment is FK.
Enrollment → Attendance	1:N	One enrollment can have multiple attendance records.	EnrollmentID in Attendance is FK.
Enrollment → AssessmentResult	1:N	One enrolled student can receive multiple assessment results.	EnrollmentID in AssessmentResult is FK.

EnrollmentID in AssessmentResult is FK.

9. How the Design Works

Remember the flow: COURSE + TRAINER → BATCH → ENROLLMENT → ATTENDANCE + ASSESSMENT_RESULT. A STUDENT enters the system through ENROLLMENT.

The Batch table stores which course is conducted and which trainer handles it. Enrollment acts as the central linking table between Student and Batch. Attendance and AssessmentResult then use EnrollmentID so that every attendance and result belongs to a valid enrolled student.

10. Expected Reports / Insights

- Course-wise student enrollment can be calculated.
- Batches with low attendance can be identified.
- High-performing and low-performing students can be identified from assessment scores.
- Student-wise training progress can be generated.
- The institute can compare course-wise student performance.
- Management can use the reports to improve training planning and monitoring.

11. Final Submission Checklist

- Cover page with First Name, Class, Roll No. and Subject
- Assignment 12 title and project objective
- Relational schema diagram with all 7 tables
- ER diagram with entities, attributes, relationships and cardinalities
- PK and FK clearly identified
- Database and table creation SQL
- Sample records for every table
- Business queries using JOIN, aggregate functions, subqueries/views/window functions where suitable
- Query outputs / screenshots added when required by the instructor
- Short explanation of relationships and query logic
- Final report / conclusion

12. Conclusion

The Training Institute Management System provides a structured relational database for managing courses, trainers, batches, students, enrollments, attendance, and assessment results. The ER model and relational schema reduce repeated data and establish clear primary-key and foreign-key relationships. SQL queries can then be used to generate useful reports for enrollment, attendance, performance, ranking, and student progress.